

Modelos de Difusión para IA Generativa\Teoría y conexión con la implementación

Documento de referencia interna — Iván Domínguez y Lucas Miranda

Abril 2026

Contents

1. Vista de pájaro	2
1.1 El problema generativo	2
1.2 Componentes del sistema	2
2. Procesos de difusión forward	4
2.1 Ecuaciones diferenciales estocásticas	4
2.2 Variance Exploding (VE) — Brownian motion	4
2.3 Variance Preserving (VP) — Ornstein-Uhlenbeck	5
2.4 Noise schedules	6
2.5 Por qué definir el kernel cerrado	6
3. Aprender a invertir: score matching	7
3.1 ¿Qué se aprende exactamente?	7
3.2 De Hyvärinen a Vincent: denoising score matching	7
3.3 La red: U-Net con condicionamiento temporal	8
4. Proceso backward (generación)	9
4.1 SDE inversa de Anderson	9
4.2 Particularización a VE y VP	9
5. Fokker-Planck y la conexión SDE-ODE	10
5.1 La ecuación de continuidad	10
5.2 Truco algebraico: del ruido al transporte	10
5.3 Implicaciones	10
6. Samplers (integradores numéricos)	11
6.1 Euler-Maruyama (EM)	11
6.2 Predictor-Corrector (PC)	11
6.3 Probability Flow ODE (PF-ODE)	12
6.4 Imputación	12
7. Métricas de calidad	13
7.1 Negative log-likelihood y BPD	13
7.2 Cómo se calcula con la PF-ODE	13
7.3 Fréchet Inception Distance (FID)	14

7.4 Inception Score (IS)	14
8. Generación controlable	15
8.1 Bayes y el score condicional	15
8.2 Classifier guidance (Dhariwal & Nichol 2021)	15
8.3 Classifier-free guidance — CFG (Ho & Salimans 2022)	15
8.4 Imputación como condicionamiento por píxeles	15
9. Pipeline completo	16
9.1 Entrenamiento	16
9.2 Muestreo	16
9.3 Evaluación	16
10. Tabla teoría → código	17
11. Hilo conductor narrativo (lectura rápida del proyecto)	18

1. Vista de pájaro

1.1 El problema generativo

Dado un conjunto de datos $\mathcal{D} = \{x^{(1)}, \dots, x^{(N)}\}$ con $x^{(i)} \in \mathbb{R}^{\mathcal{D}}$ (en nuestro caso, imágenes con $\mathcal{D} = C \times H \times W$), suponemos que cada $x^{(i)}$ es una realización i.i.d. de una distribución desconocida $p_0(x)$. El objetivo de la **IA generativa** es construir un mecanismo capaz de muestrear $x \sim p_0$, es decir, producir nuevos elementos plausibles del dataset sin tener acceso explícito a p_0 .

Los **modelos de difusión** resuelven este problema mediante una estrategia indirecta:

1. **Forward:** definir un proceso estocástico $\{X_t\}_{t \in [0, T]}$ con $X_0 \sim p_0$ que **destruye progresivamente la estructura** de los datos inyectando ruido gaussiano hasta llegar a una distribución prior p_T trivial (típicamente $\mathcal{N}(0, \sigma^2 I)$).
2. **Backward:** aprender a **invertir** ese proceso. Si sabemos invertir los infinitesimales pasos de ruido, podemos partir de $X_T \sim p_T$ y reconstruir un $X_0 \sim p_0$.

La pieza que aprende la red neuronal no es directamente p_0 , sino el **gradiente del logaritmo de la densidad** $\nabla_x \log p_t(x)$, conocido como *score*. La red lo aproxima como $s_\theta(x, t)$ a múltiples niveles de ruido t a la vez, y eso basta para invertir el proceso.

1.2 Componentes del sistema

El paquete `diffusion_lib` que hemos implementado modela cada pieza del problema como una clase abstracta, lo que permite combinarlas ortogonalmente:

```
diffusion_lib/
├─ processes/      # qué SDE usamos para el ruido          (VE, VP)
├─ schedules/     # cómo varía el ruido con t              (Linear, Cosine, Exponential)
├─ samplers/      # cómo integramos la SDE/ODE inversa    (EM, PC, PF-ODE, Imputation)
├─ metrics/       # cómo evaluamos la calidad              (BPD, FID, IS)
└─ model.py       # la fachada GenerativeDiffusionModel
```

La clase `GenerativeDiffusionModel` actúa como **fachada**: compone un proceso, un sampler y una red neuronal score, y expone tres operaciones: `compute_loss` (entrenar), `sample` (generar) y `compute_bpd` (evaluar verosimilitud).

2. Procesos de difusión forward

2.1 Ecuaciones diferenciales estocásticas

La forma más general de un proceso de difusión gaussiano es una SDE de Itô:

$$dX_t = f(X_t, t) dt + g(t) dW_t, \quad t \in [0, T]$$

donde:

- $f : \mathbb{R}^{\mathcal{D}} \times [0, T] \rightarrow \mathbb{R}^{\mathcal{D}}$ es el **coeficiente de deriva** (*drift*).
- $g : [0, T] \rightarrow \mathbb{R}$ es el **coeficiente de difusión** (escalar en nuestro caso, un múltiplo de la identidad en $\mathcal{D}$ dimensiones).
- W_t es un **proceso de Wiener** estándar $\mathcal{D}$ -dimensional, es decir, ruido blanco gaussiano integrado.

Intuición: en cada instante infinitesimal, el estado X_t se desplaza un poco según f (transporte determinista) y se le añade ruido gaussiano de magnitud $g(t)$.

La elección de f y g define el proceso. Nosotros implementamos dos: **VE** (sin drift) y **VP** (con drift que reescala los datos).

2.2 Variance Exploding (VE) — Brownian motion

Definición. Tomamos $f(X_t, t) = 0$ y $g(t) = \sigma^t$ con $\sigma > 1$ (típicamente $\sigma = 25$ para MNIST):

$$dX_t = \sigma^t dW_t.$$

Como no hay deriva, la media se conserva: $\mathbb{E}[X_t] = X_0$. La varianza, en cambio, **explota** con el tiempo. Resolviendo la SDE,

$$X_t = X_0 + \int_0^t \sigma^s dW_s,$$

y por las propiedades del integral de Itô:

$$X_t | X_0 \sim \mathcal{N}(X_0, \sigma_t^2 I), \quad \sigma_t^2 = \frac{\sigma^{2t} - 1}{2 \ln \sigma}.$$

Esto es lo que llamamos el **kernel de transición cerrado**: dado X_0 , podemos generar X_t en un solo paso, sin simular la trayectoria.

Conexión con el código (diffusion_lib/processes/ve.py):

```
class VEProcess(DiffusionProcess):
    def drift_coefficient(self, x_t, t):
        return torch.zeros_like(x_t)                # f = 0

    def diffusion_coefficient(self, t):
        return self.sigma ** t                       # g(t) = σ^t

    def mu_t(self, x_0, t):
```

```

return x_0                                     #  $E[X_t] = X_0$ 

def sigma_t(self, t):
    return torch.sqrt(
        0.5 * (self.sigma ** (2.0 * t) - 1.0) / np.log(self.sigma)
    )                                           # std de la transición

```

El método `prior_sample` devuelve $X_T \sim \mathcal{N}(0, \sigma_T^2 I)$ (la varianza al final del proceso), que es la distribución desde la que arranca el sampler en sentido inverso.

2.3 Variance Preserving (VP) — Ornstein-Uhlenbeck

Definición. Ahora hay deriva, y depende del estado:

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)} dW_t,$$

donde $\beta : [0, T] \rightarrow \mathbb{R}_{>0}$ es un *noise schedule* a definir (§2.4). Esta es una SDE lineal, cuya solución analítica es

$$X_t | X_0 \sim \mathcal{N}(\alpha_t X_0, \sigma_t^2 I),$$

con

$$B(t) = \int_0^t \beta(s) ds, \quad \alpha_t = e^{-\frac{1}{2}B(t)}, \quad \sigma_t = \sqrt{1 - e^{-B(t)}}.$$

Compactamente: $\alpha_t^2 + \sigma_t^2 = 1$ (de ahí lo de “preserva la varianza”). Cuando $B(t) \rightarrow \infty$, $\alpha_t \rightarrow 0$ y $\sigma_t \rightarrow 1$, así que $X_T \sim \mathcal{N}(0, I)$ aproximadamente.

Conexión con el código (`diffusion_lib/processes/vp.py`):

```

class VPProcess(DiffusionProcess):
    def __init__(self, schedule: NoiseSchedule, T: float = 1.0):
        self.schedule = schedule           #  $\beta(t)$ ,  $B(t)$ 
        self.T = T

    def drift_coefficient(self, x_t, t):
        return -0.5 * self.schedule.beta(t).view(...) * x_t

    def diffusion_coefficient(self, t):
        return torch.sqrt(self.schedule.beta(t))

    def mu_t(self, x_0, t):
        alpha_t = torch.exp(-0.5 * self.schedule.integral_beta(t))
        return alpha_t * x_0

    def sigma_t(self, t):
        return torch.sqrt(1.0 - torch.exp(-self.schedule.integral_beta(t)))

```

Observa que el VP **delega** en una `NoiseSchedule` para definir $\beta(t)$ y $B(t)$. Esa abstracción es la que permite intercambiar schedules sin tocar el proceso.

2.4 Noise schedules

Una `NoiseSchedule` provee dos funciones: $\beta(t)$ y su integral acumulada $B(t) = \int_0^t \beta(s) ds$. La integral aparece en $\alpha_t = e^{-B(t)/2}$ y $\sigma_t = \sqrt{1 - e^{-B(t)}}$, así que tenerla en forma cerrada nos ahorra integrar numéricamente cada vez.

2.4.1 Lineal

Entre $\beta_{\min} = 0.1$ y $\beta_{\max} = 20$:

$$\beta(t) = \beta_{\min} + (\beta_{\max} - \beta_{\min}) t, \quad B(t) = \beta_{\min} t + \frac{1}{2}(\beta_{\max} - \beta_{\min}) t^2.$$

2.4.2 Coseno

Inspirado en Nichol & Dhariwal (2021):

$$f(t) = \cos^2\left(\frac{t+s}{1+s} \cdot \frac{\pi}{2}\right), \quad \bar{\alpha}(t) = \frac{f(t)}{f(0)}, \quad B(t) = -\log \bar{\alpha}(t),$$

con $s = 0.008$ para evitar el cero exacto en $t = 0$. La derivada da

$$\beta(t) = \frac{\pi}{1+s} \tan\left(\frac{\pi}{2} \cdot \frac{t+s}{1+s}\right).$$

Importante: como $\tan(\pi/2)$ diverge en $t = 1$, se aplica un *clamp* numérico a $\beta_{\max} = 20$.

2.4.3 Exponencial

Implementación nuestra adicional, geométrica:

$$\beta(t) = \beta_{\min} \left(\frac{\beta_{\max}}{\beta_{\min}}\right)^t = \beta_{\min} e^{kt}, \quad k = \ln(\beta_{\max}/\beta_{\min}),$$
$$B(t) = \frac{\beta(t) - \beta_{\min}}{k}.$$

Las tres viven en `diffusion_lib/schedules/{linear, cosine, exponential}.py` y heredan de `NoiseSchedule`. Cualquiera puede pasarse al constructor de `VPPProcess`.

2.5 Por qué definir el kernel cerrado

El método `perturb(x_0, t)` de la clase base **es la pieza clave del entrenamiento**: con él podemos muestrear cualquier x_t directamente en un paso, sin simular toda la SDE forward:

$$x_t = \mu_t(x_0) + \sigma_t \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

```
def perturb(self, x_0, t):
    noise = torch.randn_like(x_0)
    x_t = self.mu_t(x_0, t) + self.sigma_t(t) * noise
    return x_t, noise
```

Sin esta forma cerrada, entrenar requeriría simular trayectorias enteras de la SDE en cada batch — completamente impráctico. Es lo que distingue una formulación matemáticamente “limpia” (con kernels gaussianos explícitos) de una que solo tiene la SDE.

3. Aprender a invertir: score matching

3.1 ¿Qué se aprende exactamente?

La pregunta natural es: si queremos invertir el proceso forward, ¿qué cantidad debe aprender la red neuronal?

La respuesta —no obvia— viene del teorema de Anderson (1982): para invertir una SDE de Itô basta conocer el **score** de la densidad marginal a cada tiempo,

$$\nabla_x \log p_t(x) \equiv \nabla_x \log p_t(x).$$

El score apunta hacia regiones de alta densidad. En la imagen mental: si estamos en un x ruidoso, $\nabla_x \log p_t(x)$ es la dirección en la que “la imagen se parece más a un dígito real”. El proceso backward consiste en deslizarnos por ese campo vectorial, paso a paso, mientras retiramos ruido.

3.2 De Hyvärinen a Vincent: denoising score matching

Hyvärinen (2005). Propuso aprender el score minimizando

$$J(\theta) = \frac{1}{2} \mathbb{E}_{x \sim p_t} [\|s_\theta(x, t) - \nabla_x \log p_t(x)\|^2].$$

Pero $\nabla_x \log p_t$ es desconocido. Hyvärinen demostró que esto se puede reescribir como

$$J(\theta) = \mathbb{E}_{x \sim p_t} \left[\frac{1}{2} \|s_\theta(x, t)\|^2 + \nabla_x \cdot s_\theta(x, t) \right] + \text{cte},$$

que ya no requiere conocer $\nabla_x \log p$ pero sí calcular su divergencia (impráctico en alta dimensión).

Vincent (2010). Observó que si $x = \tilde{x} + \sigma\epsilon$ con $\epsilon \sim \mathcal{N}(0, I)$, entonces

$$\nabla_x \log q_\sigma(x | \tilde{x}) = -\frac{\epsilon}{\sigma}.$$

Es decir, **el score del kernel gaussiano de transición es analítico**. Y demostró que minimizar

$$\ell_\sigma(\theta) = \frac{1}{2} \mathbb{E}_{\tilde{x}, x} \left[\left\| s_\theta(x, \sigma) + \frac{\epsilon}{\sigma} \right\|^2 \right]$$

equivale (salvo constantes) al objetivo de Hyvärinen. A esto se le llama **denoising score matching** (DSM): la red aprende a “predecir el ruido escalado”, y eso aprende el score.

Song & Ermon (2019). Extensión multi-escala — entrenar simultáneamente sobre todos los σ del schedule, ponderando con $\lambda(\sigma) = \sigma^2$:

$$\ell(\theta) = \mathbb{E}_t \mathbb{E}_{x_0, \epsilon} \left[\sigma_t^2 \cdot \left\| s_\theta(x_t, t) + \frac{\epsilon}{\sigma_t} \right\|^2 \right] = \mathbb{E}_t \mathbb{E}_{x_0, \epsilon} \left[\left\| \sigma_t s_\theta(x_t, t) + \epsilon \right\|^2 \right].$$

Conexión con el código (diffusion_lib/processes/base.py):

```

def loss_function(self, score_model, x_0, eps=1e-5):
    # 1) Sortear t uniformemente
    t = torch.rand(x_0.shape[0], device=x_0.device) * (1.0 - eps) + eps
    # 2) Muestrear x_t en cerrado
    x_t, noise = self.perturb(x_0, t)
    # 3)  $\sigma_t$  para reescalar
    sigma = self.sigma_t(t.view(...))
    # 4) Predecir score con la red
    score = score_model(x_t, t)
    # 5) Pérdida ponderada de Song
    per_sample = torch.sum((sigma * score + noise) ** 2, dim=...)
    return per_sample.mean()

```

Resumen mental: la red **aprende a predecir el ruido** que añadimos en el forward. Por eso a veces se reparametriza $\epsilon_\theta(x, t) = -\sigma_t s_\theta(x, t)$.

3.3 La red: U-Net con condicionamiento temporal

`score_model.py` define una **U-Net** ([Ronneberger 2015]) modificada para difusión:

- Bloques residuales con *group normalization*.
- **Embedding sinusoidal del tiempo** t , sumado a las activaciones de cada bloque, para que la red sepa a qué nivel de ruido está operando.
- Bloques de atención multi-cabeza a resolución intermedia para dependencias largas.

La U-Net es una elección natural porque opera en el espacio de la imagen manteniendo coherencia espacial: el cuello de botella codifica la estructura semántica de x_t , y las skip-connections preservan los detalles de alta frecuencia necesarios para predecir ϵ con precisión píxel a píxel.

4. Proceso backward (generación)

4.1 SDE inversa de Anderson

Teorema (Anderson 1982). Si $\{X_t\}$ satisface $dX_t = f(X_t, t) dt + g(t) dW_t$, entonces el proceso inverso $\{Y_s\}_{s=T-t}$ satisface

$$dX_t = [f(X_t, t) - g(t)^2 \nabla_x \log p_t(X_t)] dt + g(t) d\bar{W}_t,$$

donde $\bar{W}$ es otro Wiener y la integración va de $t = T$ a $t = 0$ (es decir, $dt < 0$ en la práctica).

La intuición física: el término $g^2 \nabla_x \log p$ “empuja” hacia las zonas densas; el término $g d\bar{W}$ aporta exploración.

4.2 Particularización a VE y VP

VE ($f = 0$):

$$dX_t = -g(t)^2 \nabla_x \log p_t(X_t) dt + g(t) d\bar{W}_t.$$

VP ($f = -\frac{1}{2}\beta x$):

$$dX_t = [-\frac{1}{2}\beta(t)X_t - \beta(t) \nabla_x \log p_t(X_t)] dt + \sqrt{\beta(t)} d\bar{W}_t.$$

Conexión con el código. El método `reverse_drift` de la clase base centraliza la fórmula de Anderson:

```
def reverse_drift(self, x_t, t, score):
    g2 = self.diffusion_coefficient(t).view(...) ** 2
    return self.drift_coefficient(x_t, t) - g2 * score
```

Sin score conocido, sustituimos $\nabla_x \log p$ por la red entrenada s_θ y obtenemos una SDE muestreable, que los samplers integran numéricamente.

5. Fokker-Planck y la conexión SDE-ODE

5.1 La ecuación de continuidad

Mientras la SDE describe trayectorias individuales (cada una aleatoria), la **ecuación de Fokker-Planck** describe cómo evoluciona la densidad $p_t(x)$ del ensemble:

$$\frac{\partial p_t}{\partial t} = -\nabla \cdot (f p_t) + \frac{1}{2} g^2 \Delta p_t.$$

Es una PDE determinista. Mismo proceso, dos descripciones equivalentes.

5.2 Truco algebraico: del ruido al transporte

Usando $\Delta p_t = \nabla \cdot (p_t \nabla \log p_t)$,

$$\frac{\partial p_t}{\partial t} = -\nabla \cdot \left[\left(f - \frac{1}{2} g^2 \nabla_x \log p_t \right) p_t \right].$$

Esto es la **ecuación de continuidad** de un fluido con velocidad

$$\tilde{f}(x, t) = f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x).$$

Resultado clave. El sistema determinista

$$\boxed{\frac{dx}{dt} = \tilde{f}(x, t),} \quad x(T) \sim p_T,$$

tiene exactamente las mismas marginales $\{p_t\}$ que la SDE original. A esta ODE se la llama **probability flow ODE** (PF-ODE).

5.3 Implicaciones

Tres consecuencias sin las cuales no se entiende el resto del proyecto:

1. **Muestreo determinista:** ya no necesitamos ruido. Una semilla x_T y un integrador ODE (Euler, Heun, RK) bastan para llegar a x_0 .
2. **Verosimilitud exacta:** como la PF-ODE es invertible, podemos usar el cambio de variable instantáneo (Chen et al., Neural ODEs 2018) para calcular $\log p_0(x_0)$ exactamente. De ahí sale el BPD.
3. **Inversión y edición:** dada x_0 , podemos ir hacia atrás integrando la PF-ODE de 0 a T , llegar a su x_T , perturbarlo y regenerar. Habilita interpolación en el espacio latente p_T .

6. Samplers (integradores numéricos)

Estamos en una situación familiar: tenemos una EDO/EDS, queremos discretizarla en N pasos $\Delta t = (\varepsilon - T)/N$ con $\varepsilon = 10^{-3}$ pequeño (no integramos hasta 0 exacto porque el score diverge).

Los tres samplers viven en `diffusion_lib/samplers/`, todos heredan de `Sampler` y exponen el método `sample(score_model, process, ...)`.

6.1 Euler-Maruyama (EM)

Discretización estocástica directa:

$$x_{n+1} = x_n + [f(x_n, t_n) - g(t_n)^2 s_\theta(x_n, t_n)] \Delta t + g(t_n) \sqrt{|\Delta t|} z_n,$$

con $z_n \sim \mathcal{N}(0, I)$.

Orden de convergencia fuerte $\mathcal{O}(\sqrt{|\Delta t|})$. Línea base. El sesgo de discretización limita la calidad cuando N es pequeño.

Código (`euler_maruyama.py`):

```
for i in range(n_steps):
    t = t_vals[i].expand(n_images)
    score = score_model(x, t)
    drift = process.reverse_drift(x, t, score) # f - g^2 * s_theta
    g_t = process.diffusion_coefficient(t).view(...)
    z = torch.randn_like(x)
    x = x + drift * dt + g_t * sqrt(abs(dt)) * z
```

6.2 Predictor-Corrector (PC)

Combina un paso de EM (predictor) con M pasos de Langevin (corrector) en el tiempo t_{n+1} :

$$x \leftarrow x + \epsilon s_\theta(x, t_{n+1}) + \sqrt{2\epsilon} z, \quad \epsilon = 2 \left(r \frac{\|z\|}{\|s_\theta\|} \right)^2,$$

con SNR objetivo $r = 0.16$ (Song 2021).

El predictor avanza la SDE, el corrector reprojeta al marginal $p_{t_{n+1}}$. Coste: $M+1$ evaluaciones de red por paso. A cambio, calidad mucho mejor con N pequeño.

Código (`predictor_corrector.py`):

```
for i in range(n_steps):
    # — Predictor (EM) —————
    score = score_model(x, t)
    x = x + process.reverse_drift(x, t, score) * dt + g_t * sqrt(|dt|) * z

    # — Corrector (M pasos de Langevin annealed) —————
    for _ in range(M):
        grad = score_model(x, t_next)
```

```

z = torch.randn_like(x)
eps = 2 * (snr * z.norm() / grad.norm()) ** 2
x = x + eps * grad + sqrt(2 * eps) * z

```

6.3 Probability Flow ODE (PF-ODE)

Sin término estocástico, integración Euler de la ODE (??):

$$x_{n+1} = x_n + [f(x_n, t_n) - \frac{1}{2}g(t_n)^2 s_\theta(x_n, t_n)] \Delta t.$$

Código (probability_flow_ode.py):

```

for i in range(n_steps):
    score = score_model(x, t)
    g2 = process.diffusion_coefficient(t).view(...) ** 2
    ode_drift = process.drift_coefficient(x, t) - 0.5 * g2 * score
    x = x + ode_drift * dt          # ¡sin término de ruido!

```

Ventajas únicas:

- **Determinismo** (mismo seed $\Rightarrow$ mismo resultado).
- **Verosimilitud exacta** (lo veremos en §7).
- Admite integradores de orden superior (Heun, RK4, DPM-Solver) que reducen drásticamente N .

6.4 Imputación

La cuarta variante (imputation.py) no es un sampler nuevo *strictu sensu*, sino un **wrap** del sampler EM con condicionamiento por máscara. Dada una observación parcial $x_\Omega = M \odot x_0$:

1. En cada paso n , calcula la versión ruidosa “esperada” del trozo conocido al nivel t_n :

$$x_t^{\text{obs}} = \alpha_t x_\Omega + \sigma_t \epsilon.$$

2. Tras la actualización backward, **sobreescribe** la parte conocida:

$$x_t \leftarrow M \odot x_t^{\text{obs}} + (1 - M) \odot x_t.$$

La parte oculta evoluciona libremente bajo la SDE inversa aprendida; la conocida permanece consistente con la observación. Al llegar a $t \approx 0$ se recupera x_0 con los píxeles faltantes inferidos.

7. Métricas de calidad

7.1 Negative log-likelihood y BPD

NLL. Para un conjunto de test $\{x^{(i)}\}_{i=1}^N$,

$$\text{NLL} = -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x^{(i)}).$$

Mide cuán bien el modelo asigna probabilidad a los datos reales. Cuanto menor, mejor.

BPD (bits per dimension). Reescala la NLL para que sea comparable entre datasets de distinta dimensión:

$$\text{BPD} = \frac{\text{NLL}}{\mathcal{D} \ln 2}.$$

7.2 Cómo se calcula con la PF-ODE

Aplicando el cambio de variable instantáneo a la ODE $\dot{x} = \tilde{f}_{\theta}(x, t)$:

$$\log p_0(x_0) = \log p_T(x_T) + \int_{\varepsilon}^T \nabla \cdot \tilde{f}_{\theta}(x_t, t) dt.$$

La traza del jacobiano se estima con **Hutchinson**:

$$\nabla \cdot \tilde{f}_{\theta}(x, t) = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0, I)} \left[\varepsilon^{\top} \frac{\partial \tilde{f}_{\theta}}{\partial x} \varepsilon \right],$$

implementado vía un **vector-Jacobian product** (`torch.autograd.grad`).

Conexión con el código (`diffusion_lib/metrics/bpd.py`):

```
for i in range(n_steps):
    x_req = x.detach().requires_grad_(True)
    score = score_model(x_req, t)
    ode_drift = process.drift_coefficient(x_req, t) - 0.5 * g2 * score

    # Hutchinson:  $\varepsilon^{\top} (\partial f / \partial x) \varepsilon \approx \text{div}(f)$ 
    epsilon = torch.randn_like(x)
    vjp = torch.autograd.grad(ode_drift, x_req,
                              grad_outputs=epsilon, retain_graph=True)[0]
    div = (epsilon * vjp).sum(dim=...)

    log_prob_delta += div * dt
    x = x + ode_drift.detach() * dt

log_p_T = process.log_prior(x, T=T)
log_p_0 = log_p_T + log_prob_delta
bpd = -log_p_0 / (D * np.log(2))
```

Atención. Para comparar con literatura (DDPM, Glow), hay que añadir el término de **dequantización**: $+\log_2 256 = +8$ bits/dim. Sin ese término, los BPD continuos pueden salir negativos (perfectamente válido para una densidad continua, pero no comparable).

7.3 Fréchet Inception Distance (FID)

A diferencia de NLL/BPD, el FID **no requiere acceso a la verosimilitud**: opera directamente sobre las muestras generadas.

Pipeline:

1. Pasar imágenes reales $\{x_i^r\}$ y generadas $\{x_i^g\}$ por InceptionV3 hasta la capa pool_3 (vector de dimensión 2048).
2. Ajustar gaussianas multivariadas a cada conjunto de features: $\mathcal{N}(\mu_r, \Sigma_r)$ y $\mathcal{N}(\mu_g, \Sigma_g)$.
3. Calcular la distancia de Fréchet (= Wasserstein-2 entre gaussianas):

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}).$$

Más bajo = mejor. Como referencia, DDPM en CIFAR-10 reporta FID ≈ 3.17 ; modelos modernos como DiT bajan de 2.0.

Conexión con el código (diffusion_lib/metrics/fid_is.py): usa torchvision.models.inception_ con pesos ImageNet.

7.4 Inception Score (IS)

$$\text{IS} = \exp\left(\mathbb{E}_{x \sim p_g}[D_{\text{KL}}(p(y|x) \| p(y))]\right),$$

donde $p(y|x)$ es la predicción de InceptionV3 sobre x . Más alto = mejor.

Mide simultáneamente:

- **Calidad:** $H(p(y|x))$ baja (cada imagen tiene una clase clara).
- **Diversidad:** $H(p(y))$ alta (las imágenes generadas cubren muchas clases).

Limitaciones: no compara con los datos reales (un modelo que memoriza train tendrá IS perfecto), y es sensible al dominio (en datasets muy distintos a ImageNet pierde sentido).

8. Generación controlable

Hasta aquí hemos generado de $p_0(x)$ libremente. La generación controlable permite muestrear de $p_0(x | y)$ con y una condición (clase, texto, observación parcial).

8.1 Bayes y el score condicional

Aplicando Bayes al score:

$$\nabla_x \log p_t(x | y) = \underbrace{\nabla_x \log p_t(x)}_{\text{score incondicional}} + \underbrace{\nabla_x \log p_t(y | x)}_{\text{score del clasificador}}.$$

El primer término ya lo aprende la red de score. El segundo requiere conocer $p_t(y | x)$, es decir, un clasificador de imágenes ruidosas.

8.2 Classifier guidance (Dhariwal & Nichol 2021)

Entrenar un clasificador auxiliar $p_\phi(y | x_t)$ sobre imágenes ruidosas y usar su gradiente como segundo término. Se introduce un peso w que controla la “fuerza” del condicionamiento:

$$\epsilon_\theta(x_t | y) = \sigma_t [\nabla_x \log p_t(x_t | y) - w \nabla_x \log p_t(y | x_t)].$$

Inconveniente: hay que entrenar **dos** modelos.

8.3 Classifier-free guidance — CFG (Ho & Salimans 2022)

Truco elegante para evitar el clasificador: entrenar **una sola red** que reciba o bien la condición y o bien una condición vacía $\emptyset$ (con probabilidad ≈ 0.1 durante training):

$$\epsilon_\theta(x_t, t) = \epsilon_\theta(x_t, t | \emptyset) + w[\epsilon_\theta(x_t, t | y) - \epsilon_\theta(x_t, t | \emptyset)].$$

$w = 0$: muestreo incondicional. $w = 1$: condicional puro. $w > 1$: condicionamiento amplificado (mejora la fidelidad a la clase a costa de diversidad).

Conexión con el código. En `generacion_condicionada/score_model.py`, el forward de la red recibe un argumento extra `class_label` que puede ser un entero (la clase) o `-1`/máscara (la condición vacía). Durante sampling, se evalúa la red dos veces y se combina linealmente.

8.4 Imputación como condicionamiento por píxeles

Caso especial del marco anterior: la condición no es una clase y , sino una observación parcial x_Ω (sub-conjunto de píxeles). El truco de proyección (§6.4) ejecuta el muestreo incondicional pero sobrescribe la parte conocida con su versión ruidosa apropiada en cada paso. No requiere reentrenar.

9. Pipeline completo

9.1 Entrenamiento

```
process = VPPProcess(schedule=CosineSchedule())
score_net = ScoreNet(...).to(device)
gm = GenerativeDiffusionModel(process, EulerMaruyamaSampler(),
                               score_net, device)
opt = torch.optim.Adam(score_net.parameters(), lr=1e-4)
```

```
for epoch in range(EPOCHS):
    for x0, _ in loader:
        loss = gm.compute_loss(x0)          # DSM ponderado
        opt.zero_grad(); loss.backward(); opt.step()
```

`compute_loss` internamente llama a `process.loss_function`, que: muestrea $t \sim U[\epsilon, 1]$, llama a `perturb` para obtener (x_t, ϵ) , evalúa la red, y devuelve la pérdida ponderada.

9.2 Muestreo

```
samples = gm.sample(
    n_images=64, img_shape=(1, 28, 28),
    n_steps=500,
    return_trajectory=True,
)
```

`gm.sample` delega en el Sampler configurado, que itera la SDE/ODE inversa partiendo de `process.prior_sample`.

9.3 Evaluación

```
bpd = gm.compute_bpd(x_test, n_steps=200, n_hutchinson=4)
fid = gm.compute_fid(real_images, fake_images)
is_mean, is_std = gm.compute_is(fake_images)
```

Cada método llama internamente a su función en `metrics/`.

10. Tabla teoría → código

Concepto teórico	Notación	Fichero	Función / clase
SDE forward	$dX_t = f dt + g dW_t$	processes/base.py	DiffusionProcess
Drift	$f(x, t)$	ve.py, vp.py	drift_coefficient
Difusión	$g(t)$	ve.py, vp.py	diffusion_coefficient
Media kernel	$\mu_t(x_0)$	ve.py, vp.py	mu_t
Std kernel	σ_t	ve.py, vp.py	sigma_t
Muestreo $x_t x_0$	$\mu + \sigma \epsilon$	base.py	perturb
Pérdida DSM	$\sum \ \sigma_t s_\theta + \epsilon\ ^2$	base.py	loss_function
Schedule lineal	$\beta(t) = \beta_{\min} + \Delta t$	schedules/linear.py	LinearSchedule
Schedule coseno	$\beta(t) \propto \tan(\cdot)$	schedules/cosine.py	CosineSchedule
Schedule exponencial	$\beta(t) = \beta_{\min} e^{kt}$	schedules/exponential.py	ExponentialSchedule
Score network	$s_\theta(x, t)$	score_model.py	ScoreNet
Reverse drift	$f - g^2 s_\theta$	processes/base.py	reverse_drift
Sampler EM	SDE inversa	samplers/euler_maruyama.py	EulerMaruyamaSampler
Sampler PC	EM + Langevin	samplers/predictor_corrector.py	PredictorCorrectorSampler
Sampler PF-ODE	$\dot{x} = f - \frac{1}{2} g^2 s_\theta$	samplers/probability_flow_ode.py	ProbabilityFlowODESampler
Imputación	proyección M	samplers/imputation.py	ImputationSampler
BPD vía PF-ODE	Hutchinson + cambio de variable	metrics/bpd.py	compute_bpd
FID	Fréchet sobre InceptionV3	metrics/fid_is.py	compute_fid
IS	$\exp \mathbb{E} D_{\text{KL}}$	metrics/fid_is.py	compute_is
CFG	$\epsilon_\emptyset + w(\epsilon_y - \epsilon_\emptyset)$	generacion_condicional.py	add_cfg_model.py
Fachada	composición	model.py	GenerativeDiffusionModel

11. Hilo conductor narrativo (lectura rápida del proyecto)

Si tuvieras que contarle el proyecto a alguien en 10 minutos:

1. **Tenemos imágenes**, no sabemos de qué distribución salen, queremos generar más.
2. **Forward**: definimos un proceso que toma una imagen y le va metiendo ruido hasta que solo queda ruido gaussiano. Hay dos sabores: VE (sin reescalar) y VP (reescalando + añadiendo). Ambos tienen kernel cerrado, así que podemos saltar a cualquier nivel de ruido sin simular.
3. **Backward**: existe una fórmula matemática (Anderson 1982) que dice cómo invertir ese proceso, pero requiere conocer el “score” $\nabla \log p_t$, que no conocemos.
4. **Aprendizaje**: aprendemos el score con una red neuronal U-Net, resolviendo un problema de **regresión**: dada una imagen ruidosa x_t , predecir el ruido ϵ que se le añadió. Esto, por una identidad de Vincent (2010), equivale a aprender el score.
5. **Generación**: con el score aprendido, integramos numéricamente la SDE inversa. Tenemos tres samplers: EM (sencillo), PC (EM con correcciones MCMC), PF-ODE (versión determinista basada en Fokker-Planck).
6. **Evaluación**: el PF-ODE permite calcular verosimilitudes exactas (BPD). Las muestras generadas se comparan con las reales mediante FID e IS sobre features de InceptionV3.
7. **Control**: con classifier-free guidance condicionamos por clase; con un truco de proyección por máscara hacemos imputación. Ambos sin reentrenar.

Y todo esto está modularizado en `diffusion_lib/`, una librería con clases abstractas para que cualquier combinación (proceso $\times$ sampler $\times$ schedule $\times$ métrica $\times$ condición) sea válida.